作品介绍文档（预提交用）

作者	叶硕迪
学校 / 专业	南京传媒学院 · 计算机科学与技术（2024 级）
联系方式	2697427690@qq.com
参赛形式	个人参赛（独立完成设计与开发）
在线演示	http://47.96.97.141:8000（无需登录）

一、项目详情及使用方式

1.1 项目是什么

科研小助手 Agent 是一个面向科研文献的个人 AI 助手。它把一篇或一批 PDF 论文，变成**可检索、可问答、可引用、可对照阅读、可分析引用关系**的结构化知识库。

解决的问题：读论文有三个反复出现的时间黑洞——

1. **找**：一篇论文 30 页，想确认「Base 模型用了几个 GPU」，得从头翻到尾；

2. **核**：用大模型总结论文时，它会编造数据，而你要逐条回原文核对；

3. **连**：同一方向的十几篇论文，引用关系散落在各自的参考文献里，看不出谁是奠基工作、谁只是衍生。

本项目针对这三件事分别给出可验证的答案：**混合检索 + 带出处的回答 + 引用图谱。**

1.2 使用方式

方式一：在线演示（推荐）

访问地址：`http://47.96.97.141:8000/docs`（接口文档）与前端页面（部署在阿里云 ECS，7×24 运行）。

典型流程（约 2 分钟）：

1. 打开「文库」页，上传 1~3 篇 PDF（支持批量、自动去重、自动版本识别）；

2. 系统自动解析 → 分块 → 向量化（一篇 15 页论文约 1 秒完成解析、2 秒完成向量化）；

3. 进入「问答」页，勾选论文范围，提问「这篇论文训练 Base 模型用了多少 GPU？」；

4. 回答的每句话后带 `[1]` `[2]` 角标，右侧同时列出被引用的原文片段、章节与页码；

5. 进入「阅读器」，可切换原文 / 双语 / 译文，整篇翻译按块缓存，翻译进度可见、可中止；

6. 进入「引用图谱」，可看到论文之间的引用网络、基石论文与孤立论文。

方式二：本地运行

```
# 后端
cd backend
python -m venv .venv && .venv\Scripts\activate
pip install -e .
uvicorn app.main:app --reload --reload-dir app --port 8000

# 前端（另开一个终端）
cd ..
```

```
python -m http.server 8080 --directory frontend
# 浏览器打开 http://localhost:8080/index.html
```

或在仓库根目录直接执行一键脚本：`.\start.ps1`（自动检查环境、启动后端、等待健康检查、打开前端）。

1.3 当前支持的范围

能力	状态
PDF 上传 / 批量 / 内容去重 / 版本识别	已实现
文本层 PDF 解析（标题、作者、摘要、章节树、参考文献）	已实现
扫描件 PDF	标记为「需校对」，OCR 未实现（见「已知局限」）
分块 + 向量化 + 混合检索（BM25 + 向量 + RRF 融合）	已实现
单篇问答 / 多篇跨文献对比 / 流式输出	已实现
中英对照阅读 + 整篇翻译（分块缓存）	已实现
引用关系抽取 + 引用图谱（入度 / PageRank / 连通分量）	已实现
划词翻译	界面已有，当前为术语表实现（未接模型）
参考文献条目与正文角标的双向跳转	未实现（数据已就绪）
导出 Markdown / HTML	未实现
arXiv 自动检索与入库	开发中

二、项目的亮点

亮点 1：抗幻觉不是一句话，而是三层可验证的机制

多数 PDF 问答产品的做法是「把片段塞给模型，让它总结」，编造无法约束。本项目做了三层：

层	机制	作用
第一层	检索无命中直接拒答，不调用大模型	没有资料时，模型的「自由发挥」是幻觉最大来源
第二层	提示词强制：只允许用给定片段作答，每个结论句必须带 `[n]` 编号	把「来源」变成硬约束而非礼貌要求
第三层	服务端校验回答里的所有编号，越界的（模型编造的）记录并清除	即使模型违规也能在落地前拦住

可验证结果：用需求文档 1.2 节给出的 5 个标准例题（作者机构、Adam 超参、Base 模型 GPU、BLEU 对比、参考文献列表）实测，同一问题各跑 2 次，**10/10 全部答对且引用来自正确论文。**

亮点 2：把「检索相关性」与「多文档覆盖度」当成两个目标

多篇论文对比时，只按相似度取前 N 条片段，会出现一个隐蔽但致命的问题：**语义上最贴近问题的那一篇会把名额吃光，另一篇一个字都进不了上下文**，模型只能回答「资料不足」。

实测问「两篇论文对 layer normalization 的使用有何不同」时，8 个片段全部来自《Layer Normalization》，另一篇完全没有进入。修复方式是**按论文配额抽取**（每篇保底 N 条），用少量相关性换取「对比双方都有材料」——因为对比任务的前提就是双方都在场。

亮点 3：界面上的每个数字都能追到数据库

开发中我们发现，首页的「解析质量表」里有图表 / 公式 / 参考文献三列，但后端并不提供这些字段，界面上一直显示 0。**那不是「还没解析」，而是界面在编造数据。**

最终选择整表移除，而不是「先留着好看」。同理，引用面板会把「没有检索到任何片段」与「检索到 N 段但模型未引用」分开显示——两者都表现为「回答不了」，但排查方向完全相反。

亮点 4：性能数据来自实测，不是估计

操作	实测结果
15 页论文解析	0.8~0.9 秒
57 块向量化（阿里云百炼 embedding）	1.7 秒
整篇翻译 51 块（并发 4）	16.6 秒
问答流式首字延迟	0.68~0.87 秒
541 页书籍解析（1860 块）	约 11 秒
被缓存命中的翻译请求	0.01 秒

亮点 5：工程上「能被验证」

- 后端 **160 个自动化测试**全部通过（含检索、RAG、去重、解析、图谱）；
- 前端 **80 项冒烟测试**（纯 Node 环境跑真实视图渲染，不依赖浏览器）；
- 线上 **34 项端到端验证**（部署脚本的一部分，每次更新后自动跑）；
- 代码规范：ruff 零告警。

亮点 6：把踩过的坑写进了文档

项目维护一份「踩坑记录」（30 余条）与「已知问题清单」（17 条），例如：

- `uvicorn --reload` 默认监视整个 backend 目录，写数据库会触发重启并掐断请求，前端表现为「连不上后端」而健康检查正常——对策是 `--reload-dir app`；
- 热重载的监视范围必须与数据目录分开；
- `localhost` 与 `127.0.0.1` 在浏览器中是两个来源，CORS 只白名单一个时所有请求被拦，报错与「服务未启动」无法区分。

三、AI 技术落地使用情况概述

3.1 用了哪些模型

用途	模型	选型理由
问答生成	DeepSeek `deepseek-chat`	中文表达自然、成本低；不使用推理模型——实测推理模型的思维链会耗尽 `max_tokens` 导致正文为空
向量化	阿里云百炼 `text-embedding-v1`（1536 维）	国内直连稳定、OpenAI 兼容接口、按量计费便宜
翻译	DeepSeek `deepseek-chat`（temperature 0.1）	术语保留要求高，低温度更稳定

3.2 怎么集成的

- 统一走 OpenAI 兼容协议（`openai` SDK + `httpx`），模型可替换；
- 密钥全部走环境变量（`.env`），仓库只提交 `.env.example` 模板；
- 调用失败分级处理：**只有连接/超时/429/5xx 才重试**（`tenacity` 指数退避），鉴权错误立刻报「密钥无效」，避免无意义重试；
- 嵌入不可用时，检索**自动降级为纯 BM25**，功能不中断。

3.3 优化策略（重点）

1. **上下文构造**：论文首页块（标题、作者、机构、摘要开头）作为固定片段强制加入，避免「第 6 作者是谁」这类元信息问题因为语义检索排名靠后而拒答；
2. **按需补充参考文献**：文献区在分块时被过滤（避免污染正文检索），当问题命中「参考文献 / references」等关键词时，再从 `citations` 表取文献列表作为一段资料追加；
3. **范围自动识别**：问题里出现某篇论文标题时，自动把该篇并入检索范围——否则默认范围会导致「答了另一篇论文」却看起来完全正常；
4. **温度取 0**：0.2 时同一问题同一资料会时而作答、时而拒答，导致能力验证不可复现；
5. **片段数按语料规模标定（6 → 8 → 12）**：语料 6 篇（1064 块）时 8 条够用；扩到 8 篇（1709 块）后 8 条已命不中「Base 模型用了多少 GPU」所需的训练细节段落，12 条才稳定，再增加答案不再变化。**片段数不是常数**，需随语料重新标定；
6. **按论文限流**：全库检索时每篇最多 3~4 条。实测某篇论文因标题与问题语义几乎一致，前 20 条命中全部来自它自己（而该篇解析质量仅 0.44、章节全丢），其它论文一条都进不来，模型只能拒答；
7. **关键词检索的信息量判据**：不只看排名第一那条命中块匹配到的词（它可能恰好只匹配 `layer` 这类高频词），而是取前 5 条命中中出现过的最强 IDF 词，避免整条关键词路被误判为噪声而跳过；
8. **参考文献片段的构造**（四个叠加缺陷逐一修正）：按问题中的英文词筛选条目（而非按顺序截断）、查找范围改为指定范围或**全库**（真正的条目常在未被语义命中的那篇文献表里）、剥离条目自带编号（避免与片段引用编号 `[15]` 混淆）、**排除问题点名论文自身的文献表**（它不会引用自己，排在前面会诱导模型判「资料不足」）；
9. **范围扩展的边界**：只在「调用方已指定范围、但锁错论文」时扩展；全库检索时**绝不收窄**——技术名词恰好等于某篇论文标题（如「Layer Normalization」）时，原逻辑会把全库搜索缩成单篇，结果是必然拒答；
10. **翻译按块缓存**：同一块只翻一次，重复请求 0.01 秒返回；重解析时**保留未变块的嵌入**，避免重复付费。

四、项目中涉及的关键技术

4.1 PDF 细粒度解析

- **书签优先**：`doc.get_toc()` 能直接给出「层级 / 标题 / 页码」，比从字号推断准确得多；无书签才回退到字号启发式；
- **竖排水印排除**：arXiv 左侧水印字号 20.0，比真标题（17.2）还大，判据用「高度 > 宽度 × 2」；
- **参考文献识别**：结合行距切段 + `[n]` 编号密度 + 年份密度 + 数字占比，实测能区分「真表格」与「参考文献区」（表格数字占比 0.30，文献区 0.05~0.09）；

- **章节继承**：解析时把当前章节标题向后传递，修复了「只有章节起始页的块才有章节名」导致 1788/2332 块丢失章节的问题。

4.2 分块与去重

- 章节感知分块，目标 700 字符、重叠 120 字符，按句子边界打包（避免把句子切断影响语义检索）；
- 自动跳过参考文献、索引、致谢等区域；
- 去重四层：SHA-1 内容指纹 → arXiv 编号 + 版本号 → 文件名规范化 → 语义近似，能区分「同一篇的不同文件名」与「同一篇的不同版本」，并提示保留哪个版本。

4.3 混合检索与 RRF 融合

- BM25（k1=1.5, b=0.75，中文按 2-gram，保留连字符复合词）+ 稠密向量；
- **RRF 只使用排名不使用分数**：两路检索的分数量纲完全不同，直接加权需要调参且不稳定，RRF 的 `1/(60+rank)` 天然免调参；
- **信息量过滤**：当 BM25 首位命中词也过于常见（相对区分度 < 50%）时，它的排名基本是噪声，直接跳过该路，避免把向量检索的正确答案挤下去。

4.4 引用图谱

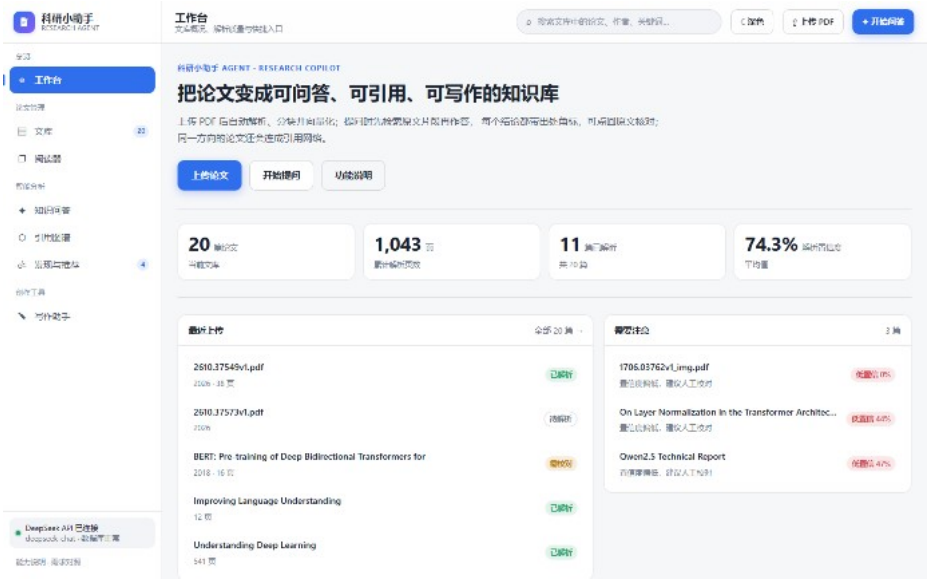
- 从参考文献文本提取 arXiv 编号（兼容新式 `1706.03762` 与老式 `cs/0101001`），优先编号精确匹配，其次用「库内标题出现在参考文献里」兜底；
- **自行实现 PageRank**（不使用 `nx.pagerank`，因为它依赖 scipy 约 40MB，且较新 Python 版本未必有预编译包），已用解析解验证：单边 A→B 得 (0.3509, 0.6491)、三节点环各得 0.3333；
- **PageRank 只在库内子图计算**：外部文献只有入边（悬挂节点），实测在 178 节点（172 个外部）的图上所有节点 PageRank 都收敛到 0.006252，完全没有区分度。

4.5 流式输出

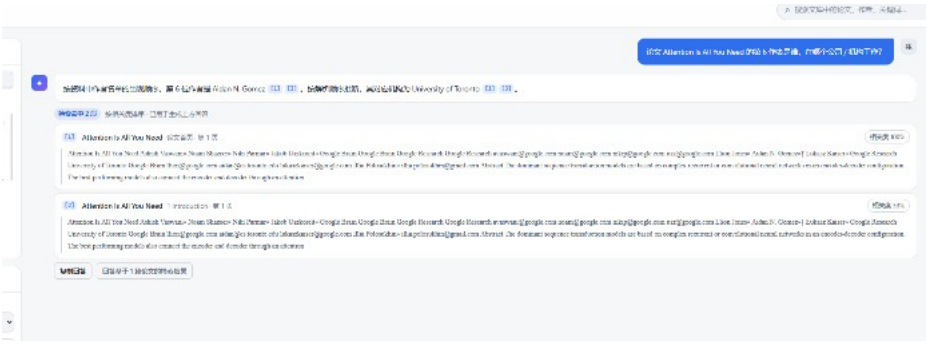
- SSE over `fetch` + `ReadableStream`（EventSource 只支持 GET，无法提交问题体）；
- 事件序列 `citations → token* → done`，先给来源再逐字给答案，首字延迟 0.7 秒左右。

五、项目相关的图片

以下截图均来自线上运行环境（20 篇真实论文，共 1116 页，非演示数据），可在 http://47.96.97.141:8000 复现。



主操作区与真实数据概览：论文数、页数、已解析数、解析置信度均读自数据库



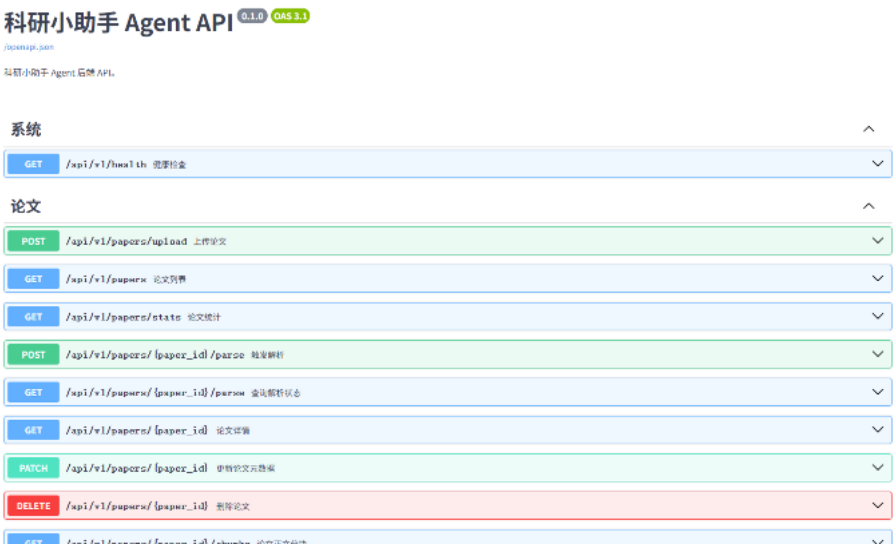
问答页：每条结论带 [n] 角标，右侧同步给出被引用的原文片段与页码



阅读器中英对照：逐段对齐，公式与代码保留原文



引用图谱：节点大小表示被引次数，右侧为 PageRank 排行



接口文档：FastAPI 自动生成的接口列表（体现工程完整度）

六、项目之前是否发表过或获奖

无。 本项目为本次比赛期间（2026 年 10 月）从零开发的原创作品，未在其他比赛投稿、未公开发表、未获奖。

项目中使用到的第三方开源库（PyMuPDF、FastAPI、SQLAlchemy、NetworkX 等）均通过依赖清单声明，未修改其源码。

七、若项目是在其他项目基础上优化，请说明优化部分

本项目为从零开发的独立项目，不是对既有项目的二次开发。

为便于评审判断原创范围，说明如下：

部分	来源
前端界面（8 个页面，原生 JS，零框架依赖）	全部自研
PDF 解析、分块、去重、版本识别	全部自研（使用 PyMuPDF 作为底层 PDF 读取库）
BM25 实现、RRF 融合、检索调度	全部自研（未使用 Elasticsearch / LangChain 等成熟检索框架）
PageRank	自行实现（未使用 NetworkX 的 `pagerank`，原因见 4.4）
RAG 提示词、三层抗幻觉、引用校验	全部自研
翻译服务与缓存	全部自研
部署脚本、线上验证脚本	全部自研

未使用任何「开箱即用」的 RAG 框架（如 LangChain / LlamaIndex），检索、融合、提示词、校验全部自己实现。这样做的工作量更大，但每一步都可解释、可调参、可验证——这也是本项目能把「为什么这样设计」写清楚的原因。

附：已知局限（如实列出）

项	现状	改进路径
扫描件 OCR	仅标记「需校对」	接入视觉模型做页面识别
表格 / 公式提取	表格检测能力弱，公式按纯文本展示	评估 `pymupdf_layout` + KaTeX
版式还原（单双栏）	按结构化文本 + 章节导航呈现	按检测到的栏边界切分后重排
引用角标 ↔ 参考文献双向跳转	数据已就绪，接口未做	`GET /papers/{id}/references` + 正文角标可点击
划词翻译	当前为术语表实现	接入翻译接口
导出 Markdown / HTML	未实现	由分块与译文生成
arXiv 自动检索与入库	开发中	`arxiv` 依赖已就绪